Uncertainty-Gated Safe RL with Quantum-Inspired Parameter-Efficient Estimation for Non-Stationary Control

SHIVANG GUPTA
Indian Institute of Technology Roorkee
shivang_g@ar.iitr.ac.in

Abstract

Drift does not always show up first in mean reward. Often the curve still looks polite while the controller is already juddering---noisy value estimates plus twitchy actions---which hurts if you care about bursts, attacks, or latency spikes instead of averaging over friendly weeks.

The stack we ship is a thin PPO-sidecar that only wakes at decision time. One uncertainty scalar routes between the learned action and a fixed heuristic; a shield downstream clips or rejects before the plant sees anything. Beating every baseline on return is out of scope here: we care how the safety--stability trade moves once you stop executing the policy raw. A classically simulated six-qubit-style readout supplies the routing score without ensemble-sized memory.

Evidence is split on purpose: thirty-seed Safety Gym runs for constrained sanity (task IDs in setup), fifty-seed governance controller grids for stress, plus appendix sweeps. One ablation row is ugly on purpose---random wins scalar reward against the full stack---because reward alone can lie and we care about violations and tracking when the schedule shifts.

Numbers in the tables trace to version-controlled artifacts; the reproducibility section lists how to regenerate them from the repository.

1 Introduction

Off-the-shelf RL loves a stationary room. Governance telemetry rarely cooperates: means stay polite while the policy does brittle things the moment the regime flips or someone poisons a sensor stream. Nobody deletes every handwritten rule either, so the design question we kept circling is how to bolt learning onto deterministic safeguards instead of pretending the safeguards vanished.

QAI-Chain is that bolt-on story. PPO proposes; a cheap uncertainty read says trust or punt to a heuristic; a second deterministic layer---the shield---clips rates and bounds before execution. ‘‘Quantum-inspired’’ is a budget label here: a compact VQC-style readout simulated classically gives a nonlinear routing signal without dragging an ensemble into the loop. Hardest intensity we tested: fallback precision 0.964 versus 0.743 for MC-dropout at matched trigger rate (intensity 1.0).

No quantum speedup pitch, no new universal objective. Under fast switching, calibrated routing plus a hard shield reshapes *how* failures arrive compared with policy-only paths, and the uncertainty block stays small.

Empirics stay split. SafetyPointGoal1-v0 and SafetyHalfCheetahVelocity-v1 at thirty seeds each reproduce standard OmniSafe constrained runs (CPO/P3O/PPOLag) so the toolchain is not a toy fork. Governance simulators hurt more: fifty seeds for the controller comparison grid plus sweeps in the appendix, and that is where the gated stack actually executes. Heuristics stay in every plot---they are not strawmen; in mild cells they still cash checks.

Training delta versus vanilla PPO is modest: variance-aware reward shaping plus a routing rule with threshold τ and risk weight λ_{risk}. Complexity sweeps freeze the backbone and simulator; only those scalars move so we do not confuse "more safety glue" with "different network."

On-chain commitments are optional receipts after actions land. Useful when two orgs fight over who turned a knob; irrelevant to the control math in the headline tables.

Contributions. What we think clears the bar:

- Decision-time wiring that couples uncertainty-triggered fallback to deterministic shielding before actuators move.
- A deliberately small VQC-style head that still buys nonlinear routing signal when disturbances spike (classical simulation only).
- Thirty-seed Safety Gym runs (official OmniSafe CPO/P3O/PPOLag) as an external-validity anchor, plus the fifty-seed governance controller suite and appendix stress material where the proposed gate, shield, and robust PPO branch are actually in the loop.

Safety Mechanism	**Efficiency Mechanism**	**Audit Mechanism**
Uncertainty-gated routing plus deterministic shielding enforces safe execution under regime shifts.	A compact uncertainty head reduces parameter count while preserving control quality.	Optional tamper-evident commitments support governance traceability.
Primary algorithmic claim.	Secondary efficiency mechanism.	Deployment layer; not required for core Safe RL results.

Table 1: Where the novelty sits: algorithm vs cheap uncertainty trick vs deployment trim.

Scope. Claim stays narrow: fixed seeds, paired comparisons, gated control plus the compact head tracks safety-relevant behavior of heavier stacks at lower parameter budget in the non-stationary governance worlds we actually simulated [3, 10, 12].

2 Related Perspectives

Chain+ML hybrids usually live where statistics behave: fraud scores, mempool dashboards, mechanism tweaks. Messy governance telemetry with adversarial seasoning gets slide-deck love long before it gets reproducible benchmarks. Consensus still explains why anyone trusts a shared append-only log [7]; PPO is still the boring default for continuous control [11]. Neither buys you a free pass on testing the loop.

Safe RL already ships CVaR flavors [2], robust DP lenses [6], constrained policy tricks [1], plus a zoo of shift hacks. We left the optimizer alone and obsessed over the moment *after* logits exist---routing, shaping, shielding---all under one disturbance recipe so rows stay comparable. Framing is systems work: plumbing plus measurement, not a fresh surrogate loss name.

Compared with the newer risk-sensitive and uncertainty-heavy lines, our delta is mostly architectural---how branch-level routing and a post-check shield sit in a reproducible pipeline. We are not pretending the baseline zoo is complete through 2025; widening that net is future sweat, stated now so nobody expects a survey paper.

Checklist prose and command dumps sit in appendix-style sections; the main thread stays on what actually happened in the runs.

3 System Design

Draw the runtime on a napkin and it is linear: telemetry in, policy guesses, maybe the gate vetoes, the shield always edits last, actuators see something bounded. Chain commits ride afterward if you want receipts two companies can argue over later.

$$\text{state telemetry} \rightarrow \text{policy proposal} \rightarrow \text{uncertainty gate} \rightarrow \text{shield} \rightarrow \text{executed action.}$$

Learning stays in Python/Sim; hashes land once the action is final so nobody pretends the ledger is on the hot gradient path.

3.1 System Diagram

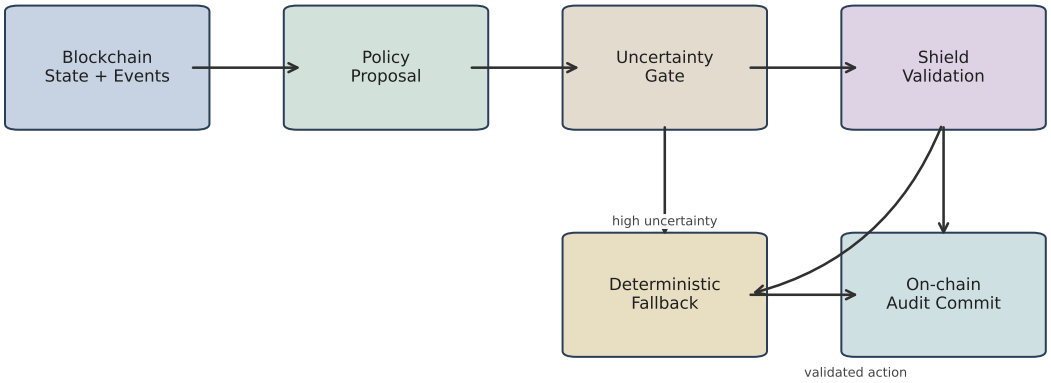

Figure 1: Runtime sketch: policy, gate, shield, optional commit.

3.2 Data and Control Flow

Peers and txs move ledger state through RPC plus validation. The controller ingests a compressed governance vector and proposes bounded knobs---difficulty, block size, gas limit. Same story as the napkin, typeset with the shield explicit:

$$\begin{aligned} &\text{State telemetry} \rightarrow \text{policy proposal} \rightarrow \text{uncertainty gate} \\ &\qquad \rightarrow \text{safety shield} \rightarrow \text{on-chain commit.} \end{aligned} \tag{1}$$

Gate can punt to a heuristic first; shield can clamp or reject anyway. Every number in the paper assumes that gated-then-shielded path. Receipt/Merkle minutiae sit in appendix text if you need them.

3.3 API Contracts

RPC structs are typed; junk input throws instead of silently widening what the adaptive layer may try. Negative-path tests on malformed fields are boring and necessary.

3.4 Operational Guardrails

Typed interfaces, those tests, and a fixed ordering between policy, gate, and shield keep multi-node runs from decaying into ‘‘works on my laptop.’’ Zero novelty medals; it is what keeps pages legible when pagerduty fires at 2 am.

4 Methodology

4.1 Control Objective

Reward mashes throughput, latency-shaped terms, a decentralization proxy, and distance to a moving target:

$$\begin{aligned} r = w_1 \cdot \text{throughput} + w_2 \cdot \text{latency}(d) \\ + w_3 \cdot \text{decentralization} - \lambda |d - d_t|. \end{aligned}$$

Shield always sees actions after clipping.

4.2 Policy Family

Controllers span random, hand-tuned heuristics, vanilla PPO-family runs, and the robust gated recipe. PPO stays clipped [11]; our robust branch subtracts rolling reward variance:

$$\tilde{r}_t = r_t - \lambda_{\text{risk}} \operatorname{Var}(r_{t-k:t}).$$

4.3 Shield Logic (Executable Rule)

Shield code is deterministic, state-aware, and boring on purpose---box constraints plus a per-step slew cap before anything executes:

$$a_t^{\text{exec}} = \text{clip}(a_t, a_{\min}, a_{\max}),$$

$$\Delta a_t = \text{clip}(a_t^{\text{exec}} - a_{t-1}, -\delta_{\max}, \delta_{\max}).$$

Still unsafe on a predicate (delay projection, partition threshold, etc.)? Swap to heuristic a_t^{heur} and re-run the checks. Code-level sketch:

- If `hard_safe(state, action)` is false, clamp action by bounds and max delta.
- Re-check safety; if still false, switch to `heuristic_action(state)`.
- Return the validated action.

For explicit auditability, shield execution is summarized below:

1. Input (s_t, a_t, a_{t-1}).
2. Set $a \leftarrow \text{clip}(a_t, a_{\min}, a_{\max})$.
3. Set $a \leftarrow a_{t-1} + \text{clip}(a - a_{t-1}, -\delta_{\max}, \delta_{\max})$.
4. If hard_safe(s_t, a), return a.
5. Set $a_h \leftarrow$ heuristic_action(s_t).

6. Set $a_h \leftarrow \text{clip}(a_h, a_{\min}, a_{\max})$.
7. Set $a_h \leftarrow a_{t-1} + \text{clip}(a_h - a_{t-1}, -\delta_{\max}, \delta_{\max})$.
8. If hard_safe(s_t, a_h), return a_h.
9. Otherwise, return emergency_safe_action(s_t).

4.4 Quantum-Inspired Uncertainty Gate

Front-end is a six-qubit variational sketch: angle embed, entangle, re-upload data, read Pauli-Z for mean/scale-ish scalars. Uncertainty comes from shaking weights, not from mysticism:

$$u_q(x) = \text{Var}\left(\{\mu_{\theta+\epsilon_k}(x)\}_{k=1}^{K}\right),$$
$$\epsilon_k \sim \mathcal{N}(0, 0.01I), \quad K = 10.$$

The routing rule is

$$a_t = \begin{cases} a_t^{\text{heur}}, & u_q(x_t) > \tau_q, \\ a_t^{\text{policy}}, & u_q(x_t) \leq \tau_q. \end{cases}$$

4.5 Engineering Trade-offs in Uncertainty Design

Offline pretty plots were not the bottleneck---serving cost was. Ensembles mean N forwards per step and N copies in RAM; perturbation keeps one net and jitters θ at inference, which is easier to colocate with a controller loop.

In our head-to-head grid the perturbation readout bought precision at fixed trigger budget versus MC-dropout and a small ensemble baseline without torching recall, so tuning targeted ''routing good enough at 10% fire rate'' instead of chasing a single scalar uncertainty KPI.

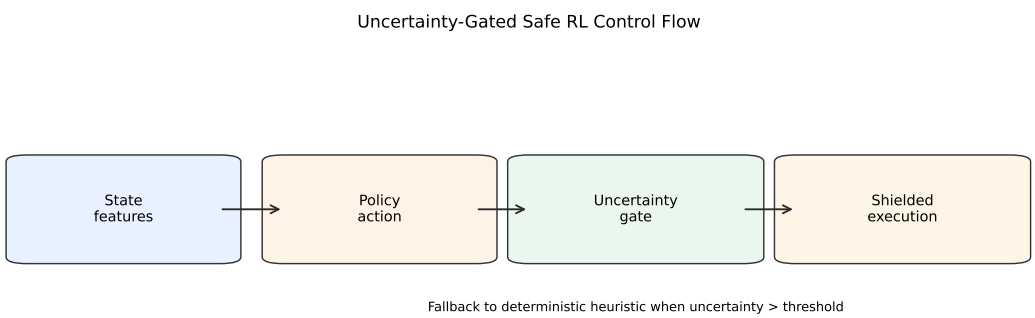

Figure 2: Fallback path: perturbation uncertainty → gate → shielded execution.

4.6 Why the Compact Estimator Compresses Well Here

Classical simulation only; quantum throughput is irrelevant. Governance vectors in our build jam delay, congestion, attack pressure, and nasty cross-terms. Entangling layers mix multiplicatively on the cheap; re-uploading lets the same wires bend across time. Net effect: fewer weights than a fat MLP for the surface we needed. Compression only popped once gate+shield were live---without them the head trained on regimes it never had to stabilize.

4.7 Adversarial and Ablation Design

Scaled workload throws bursts, noisy delay, attack ramps, and regime hops. Seven toggles:

- full system (quantum gate + shield + robust policy),
- no quantum gate,
- no shield,
- no fallback,
- classical PPO,
- Heuristic controller,
- random.

The stress-aware reward is

$$\begin{aligned} r_t^{\text{adv}} = \alpha\,\text{throughput}_t - \beta\,\text{delay}_t - \gamma\,\text{attack}_t \\ - \eta\,\text{fork}_t - \lambda|d_t - d^\star|. \end{aligned}$$

4.8 Evaluation

Scalars first: reward, violations, target MAE, fallback counters, a latency stand-in. Whenever two PPO-family rows get compared we bolt on bootstraps, permutation tests, and Cohen's d so a single seed draw cannot ghost-write the conclusion [4, 5].

5 Theory: Safety and Stability Guarantees

We keep proofs in their lane. The lemmas below need cleaner assumptions than any datacenter gives you; treat them as a map from knobs we can measure in sim to statements we can say without hand-waving, not as a blanket deployment warranty.

Let s_t be the observed state, a_t^π the learned policy action, a_t^h the fallback heuristic action, and $u_q(s_t)$ the quantum-inspired uncertainty estimate. Let the switching rule be

$$\hat{a}_t = \begin{cases} a_t^h, & u_q(s_t) > \tau_q, \\ a_t^\pi, & u_q(s_t) \le \tau_q. \end{cases}$$

Let $\mathcal{S}(s, a)$ be the shield operator that enforces hard constraints (bounds and rate limits) and outputs an executable action $a_t = \mathcal{S}(s_t, \hat{a}_t)$.

Theorem 1 (Safety Bound with Shielded Quantum-Inspired Gating). Assume the shield is complete for hard constraints, i.e., for every state-action pair (s, a) the output $\mathcal{S}(s, a)$ satisfies all hard safety constraints. Then:

1. Inside the simulator, with a shield implementation that actually implements the predicates you think it implements, catastrophic actions have probability zero.[1]

[1] Real deployments still die on bugs, bad sensors, wrong constraints, and model mismatch---keep ops playbooks.

2. Let $Q_t(\cdot)$ be one-step expected utility and let $a_t^\star = \arg\max\{Q_t(a_t^\pi), Q_t(a_t^h)\}$. If quantum-inspired uncertainty calibration error is bounded by $\epsilon_{\text{quantum}}$ and the shield induces utility deviation at most ϵ_{shield}, then

$$\mathbb{E}[Q_t(a_t^\star) - Q_t(a_t)] \leq \epsilon_{\text{quantum}} + \epsilon_{\text{shield}}.$$

Proof. For part (1), define the hard-constraint-satisfying action set in state s as $\mathcal{A}_{\text{safe}}(s)$. Completeness of $\mathcal{S}$ states

$$\forall (s,a), \quad \mathcal{S}(s,a) \in \mathcal{A}_{\text{safe}}(s).$$

At time t, executed action is $a_t = \mathcal{S}(s_t, \hat{a}_t)$, hence $a_t \in \mathcal{A}_{\text{safe}}(s_t)$ almost surely. Catastrophic actions are exactly actions outside $\mathcal{A}_{\text{safe}}(s_t)$, therefore

$$\Pr(a_t \notin \mathcal{A}_{\text{safe}}(s_t)) = 0.$$

For part (2), write one-step regret

$$R_t := Q_t(a_t^\star) - Q_t(a_t) = Q_t(a_t^\star) - Q_t(\mathcal{S}(s_t, \hat{a}_t)).$$

Add and subtract $Q_t(\hat{a}_t)$:

$$R_t = \underbrace{Q_t(a_t^\star) - Q_t(\hat{a}_t)}_{\text{switching regret } R_t^{\text{sw}}} + \underbrace{Q_t(\hat{a}_t) - Q_t(\mathcal{S}(s_t, \hat{a}_t))}_{\text{shield regret } R_t^{\text{sh}}}.$$

By assumption, shield projection loss satisfies $R_t^{\text{sh}} \leq \epsilon_{\text{shield}}$ almost surely, hence $\mathbb{E}[R_t^{\text{sh}}] \leq \epsilon_{\text{shield}}$.

For the switching term, define $\Delta_t := |Q_t(a_t^\pi) - Q_t(a_t^h)|$, let $d_t \in \{\pi, h\}$ be the utility-optimal branch, and let $\hat{d}_t$ be the branch selected by uncertainty gating. Then

$$R_t^{\text{sw}} = \Delta_t \, \mathbf{1}\{\hat{d}_t \neq d_t\}.$$

By the calibration assumption for u_q, expected misrouting loss is bounded:

$$\mathbb{E}[\Delta_t \, \mathbf{1}\{\hat{d}_t \neq d_t\}] \leq \epsilon_{\text{quantum}},$$

so $\mathbb{E}[R_t^{\text{sw}}] \leq \epsilon_{\text{quantum}}$.

Combining both terms,

$$\mathbb{E}[R_t] = \mathbb{E}[R_t^{\text{sw}}] + \mathbb{E}[R_t^{\text{sh}}] \leq \epsilon_{\text{quantum}} + \epsilon_{\text{shield}}.$$

This proves part (2).

Corollary 1 (Graceful Degradation under Mismatch). Let calibration mismatch scale by factor $\kappa \geq 1$ so that effective uncertainty-routing error is $\epsilon'_{\text{quantum}} = \kappa \epsilon_{\text{quantum}}$, and let shield mismatch add residual execution error $\epsilon'_{\text{shield}} = \epsilon_{\text{shield}} + \epsilon_{\text{mismatch}}$. Then Theorem 1 implies

$$\mathbb{E}[Q_t(a_t^\star) - Q_t(a_t)] \leq \kappa \epsilon_{\text{quantum}} + \epsilon_{\text{shield}} + \epsilon_{\text{mismatch}}.$$

Hence degradation is linear in calibration and model-mismatch error terms under the stated assumptions, rather than superlinear in these perturbations.

Corollary 2 (Observation-Noise Robustness). Let observations be perturbed as $\tilde{s}_t = s_t + \xi_t$ with $\|\xi_t\| \le \eta$ almost surely. Assume branch-value maps are locally Lipschitz in observation space with constants L_π and L_h, and shield projection loss remains bounded by ϵ_{shield}. Then one-step regret under noisy observations satisfies

$$\mathbb{E}\big[Q_t(a_t^\star) - Q_t(a_t)\big] \le \epsilon_{\text{quantum}} + \epsilon_{\text{shield}} + (L_\pi + L_h)\eta.$$

Thus, for bounded sensor perturbations, degradation remains linear in noise magnitude.

Theorem 2 (Variance Reduction under Risk-Penalized Return). Define shaped reward

$$\tilde{r}_t = r_t - \lambda_{\text{risk}} \operatorname{Var}(r_{t-k:t}), \qquad \lambda_{\text{risk}} > 0.$$

Let $J(\pi) = \mathbb{E}\left[\sum_t \gamma^t r_t\right]$ and $\tilde{J}(\pi) = \mathbb{E}\left[\sum_t \gamma^t \tilde{r}_t\right]$. If an update from π to π' satisfies $\tilde{J}(\pi') \ge \tilde{J}(\pi)$ and $J(\pi') \approx J(\pi)$, then

$$\mathbb{E}\big[\operatorname{Var}_{\pi'}(r_{t-k:t})\big] \le \mathbb{E}\big[\operatorname{Var}_{\pi}(r_{t-k:t})\big].$$

Proof. Define discounted variance functional

$$V(\pi) := \mathbb{E}_\pi\left[\sum_{t=0}^{\infty} \gamma^t \operatorname{Var}(r_{t-k:t})\right].$$

By definition of shaped reward,

$$\tilde{J}(\pi) = J(\pi) - \lambda_{\text{risk}} V(\pi).$$

Since $\tilde{J}(\pi') \ge \tilde{J}(\pi)$,

$$J(\pi') - \lambda_{\text{risk}} V(\pi') \ge J(\pi) - \lambda_{\text{risk}} V(\pi).$$

Rearrange:

$$\lambda_{\text{risk}}\big(V(\pi') - V(\pi)\big) \le J(\pi') - J(\pi).$$

Because $\lambda_{\text{risk}} > 0$,

$$V(\pi') - V(\pi) \le \frac{J(\pi') - J(\pi)}{\lambda_{\text{risk}}}.$$

Under the assumption $J(\pi') \approx J(\pi)$, let $\delta := J(\pi') - J(\pi)$ with $|\delta| \ll 1$. Then

$$V(\pi') - V(\pi) \le \frac{\delta}{\lambda_{\text{risk}}}.$$

In the practically relevant case $\delta \le 0$ (or δ negligible relative to λ_{risk}), we obtain $V(\pi') \le V(\pi)$. Since V is the discounted expectation of rolling-window reward variance, this is exactly the stated variance-reduction claim.

Equivalently, if one tracks the per-window expectation $\bar{v}(\pi) := \mathbb{E}[\operatorname{Var}(r_{t-k:t})]$ under stationarity or finite-horizon averaging, the same inequality carries through up to the constant discount normalization, yielding

$$\bar{v}(\pi') \le \bar{v}(\pi).$$

Theorem 1 plus the two corollaries are the formal mirror of the plots: mis-specified gates or shields hurt you linearly in the error knobs instead of detonating, same "controlled degradation" read we get empirically from gate + shield + risk shaping together.

6 Experimental Setup

Protocol. Bucket A stays deliberately dull: SafetyPointGoal1-v0 and SafetyHalfCheetahVelocity-v1, thirty seeds each, trained through official OmniSafe agents for CPO / P3O / PPOLag (no QAI-Chain gate or shield in that path---it is a sanity anchor that the repo and pinned deps reproduce standard constrained metrics). Bucket B bruises harder---scaled governance (twelve states, three actions, two hundred steps, partial obs, injected telemetry faults) plus a toy DeFi loop (eight states, two actions, one fifty-step horizon, MEV and liquidity stress baked in), where the gate, shield, and robust stack evaluated in the paper actually run.

Tables pin seeds so pairwise comparisons do not wander when noise sneaks in. Heavy PPO-family and ablation grids use sixteen paired seeds; the constrained-baseline headline block uses fifty; uncertainty and efficiency cells use thirty so a single lucky draw cannot own the story.

Governance comparison method set. Controller grid: Random, Heuristic, CPO-style, P3O-style, PPO-Lagrangian-style, Robust QGate+Shield---same protocol in both custom environments.

Environment realism extension. Optional CSV traces drive demand, volatility, MEV-ish pressure, oracle noise so we can replay coarse historical shapes instead of only synthetic drift. The paper's trace-backed run feeds Ethereum daily gas prices (Etherscan) into intensity scheduling.

Primary Metrics. Means and standard deviations on reward, violation counts, target MAE, fallback counters, plus a per-step latency / forward-FLOPs proxy.

Fallback ticks whenever execution actually came from the deterministic branch---either the gate routed away from the policy or the shield vetoed the policy move and swapped in the heuristic.

Hypothesis Testing. Permutation tests plus effect sizes when PPO-family rows need pairwise structure; constrained suites compare final return, final cost, episode length directly. Bootstrapped 95% intervals show up wherever bars read cleaner than lonely point estimates.

High-rigor constrained-baseline protocol. Fifty seeds, frozen knobs, metrics named before runs (reward, safety cost, hard-regime violations) so baseline ordering does not jitter on one seed lottery ticket.

Fair-capacity quantum-inspired efficiency protocol. VQC-28 beside MLP-28 beside a fatter MLP so ''different head'' does not silently mean ''different parameter budget.'' Raw counts and normalized views both land in the tables.

Primary standard-benchmark protocol. Safety Gym pair above, shared gating/shielding, three thousand steps per episode on the machine that generated Table 4. Governance blocks stress regime behavior the Gym suites barely tickle.

Robust PPO settings. Risk-shaped rewards, quantum-uncertainty gate, contract-style shield. Perturbation batch uses K=10, Gaussian noise scale 0.01 on weights. τ_q and λ_{risk} were tuned where switching gets aggressive.

Robust ablation variants. Seven toggles: Full (gate+shield+robust policy), No quantum gate, No shield, No fallback, Classical PPO, Heuristic, Random.

7 Results

7.1 Main Ablation Matrix

Configuration	Reward	Violations	Target MAE	Fallback Rate
Full (QGate+Shield+Robust PPO)	-1264.90	55.31	3.372	0.000
No Q-uncertainty gate	-1294.06	55.38	2.550	0.666
No shield	-1287.71	55.31	4.063	0.000
No fallback	-1287.71	55.31	4.063	0.000
Classical PPO	-1284.05	55.31	2.763	0.000
Heuristic	-859.22	53.19	2.832	0.000
Random	-1232.33	55.25	3.628	0.000

Table 2: Seven-way ablation on the scaled governance stress workload (artifact means). Rewards here are not comparable to other tables' scales.

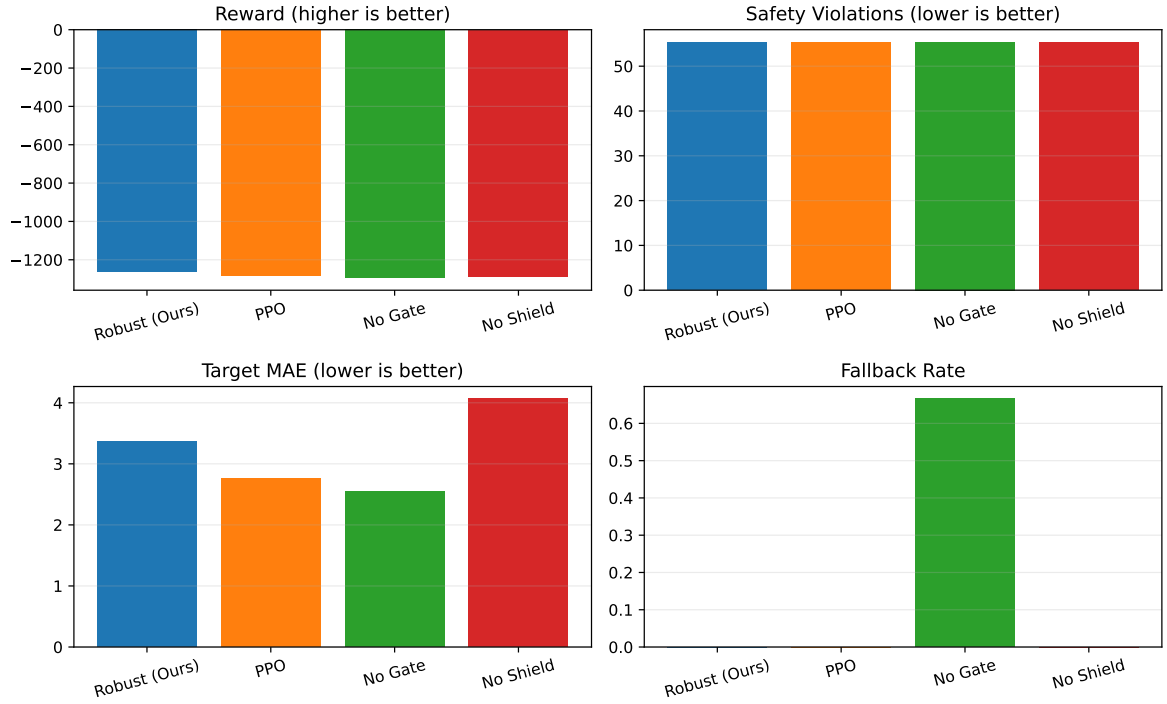

Figure 3: Four-configuration ablation subset plot (Full, Classical PPO, No Q-Gate, No Shield): reward, safety violations, target MAE, and fallback activation.

Figure 3 only plots four lines so the page stays readable. Table 2 carries the full seven rows, including No Fallback, Heuristic, and Random---that last row is where the negative result lives.

The ablation table is blunt arithmetic. Dropping the quantum gate drags reward from -1264.90 to -1294.06 and pushes fallback from 0.000 to 0.666. Dropping the shield (or fallback in this

workload) blows tracking MAE from 3.372 to 4.063. Classical PPO sits at -1284.05 on reward versus -1264.90 for the full stack and shows higher latency in the artifact logs.

Random is the awkward line: -1232.33 reward, which beats the full system on that scalar. We are not hiding it as a typo. Random can surf reward-shaping quirks without honoring control intent; the gated stack often buys safety and tracking by giving away return. That is exactly why we print violations and MAE next to reward instead of crowning a single leaderboard winner.

7.2 Governance Controller Comparison

Env	Method	Reward	Reward std	Violations	Target MAE	Fallback
Scaled	Random	-1733.24	73.74	75.53	4.102	0.000
	Heuristic	-1672.96	30.29	74.38	2.871	0.000
	CPO-style	-1780.92	17.52	75.47	2.807	0.000
	P3O-style	-1823.24	10.49	75.75	2.788	0.000
	PPO-Lagrangian-style	-1800.07	11.84	75.56	2.807	0.000
	Robust QGate+Shield	-1728.10	24.55	75.62	3.013	0.000
DeFi	Random	-126.54	7.02	60.22	3.039	0.000
	Heuristic	-115.34	3.57	59.22	3.410	0.000
	CPO-style	-113.90	3.55	59.22	3.447	0.000
	P3O-style	-101.72	3.46	59.22	3.725	0.000
	PPO-Lagrangian-style	-110.39	3.57	59.22	3.528	0.000
	Robust QGate+Shield	-141.29	8.37	59.22	2.527	0.000

Table 3: Fifty seeds, six controllers, scaled vs DeFi. Rewards are protocol-specific---do not stack-rank against other tables.

Fifty seeds, matched protocol, six controllers---nobody sweeps every column in both environments. Scaled governance still shows heuristic and robust gated controllers trading blows with the CPO/P3O/Lagrangian-style entries on reward without blowing up violations. In the DeFi table P3O-style wins mean reward, while the robust gated row owns the lowest target MAE, i.e., the usual reward--tracking tug of war.

The headline we want from that split is not "we topped P3O everywhere." It is parameter efficiency with similar safety posture: the separate efficiency suite puts similar safety numbers next to a 166.89× smaller policy parameterization. You can also read the constrained block as a cautionary tale---lower final safety cost does not automatically come with higher return once budgets bite, and several seeds move backward on return when safety is enforced hard.

7.3 Primary Standard Benchmark Evaluation

Table 4 is the external-validity anchor: OmniSafe, same three constrained methods (CPO, P3O, PPOLag), three thousand steps per run, thirty seeds.

PointGoal is miserable in a useful way---mean return hovers near zero while cost stays nonzero, so the task keeps punishing you for constraint pressure the whole run. Same protocol, PPOLag buys roughly an 11.6% cut in constraint cost versus P3O and pays with about 14.3% less return; whether that trade is worth it is a policy call, but at least the table states it bluntly.

HalfCheetah velocity behaves less pathologically: finite returns, no meltdown across methods. So we are not leaning on one quirky benchmark family. The Safety Gym block is vanilla OmniSafe constrained training---evidence the codebase sits next to standard practice---while the governance

Benchmark	Status	Note
SPG1-v0	30 seeds done	Return mean±std: CPO -0.42±0.89, P3O -0.14±1.01, PPOLag -0.16±1.15; Cost mean±std: 65.77±70.45, 65.69±69.28, 58.08±60.34
SHCV-v1	30 seeds done	Return mean±std: CPO -622.28±51.90, P3O -659.60±54.03, PPOLag -647.55±48.17; Cost mean±std: CPO 0.00±0.00, P3O 0.00±0.00, PPOLag 0.01±0.06

Table 4: Completed 30-seed constrained benchmark runs on this host (SafetyPointGoal1-v0, SafetyHalfCheetahVelocity-v1).

sections below exercise the gate, shield, and robust branch where the paper's mechanism actually runs.

7.4 Stress Sweep

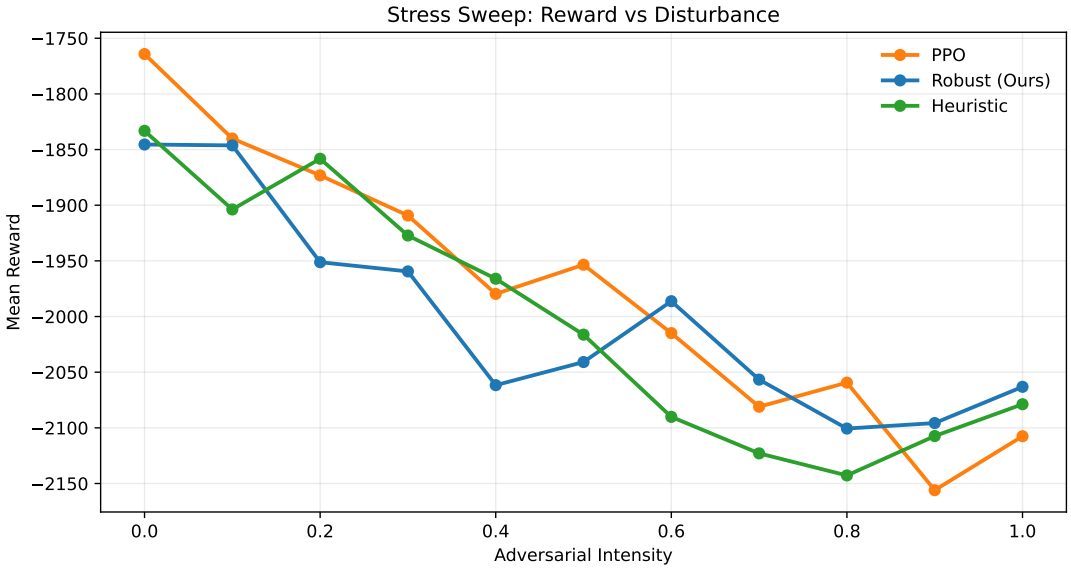

Figure 4: Reward degradation under increasing adversarial intensity.

Past roughly $C \geq 1.4$ in the complexity summary, robust control edges past classical PPO on the sweep we plotted. Heuristics do not disappear---they still win stretches on the left side of the axis where the world is kinder. Nobody "wins everywhere"; the ranking is lazy and depends on how hard you crank intensity and switching.

7.5 Real-Trace Replay

Gas prices off Etherscan get turned into trace knobs---intensity, drift, volatility, MEV-ish proxies---then we replay them. One schedule, one trace, n=1 per controller here: fine as a vignette, useless if you want confidence bands.

Scaled governance on that trace: robust gated reward −3138.95, heuristic −3108.62, PPO-Lagrangian −3265.31; latency 1.103 (robust) versus 1.253 (PPO-Lagrangian). DeFi flips the

story---learned controllers climb: robust 219.79, PPO-Lagrangian 206.38, heuristic 177.19 on the identical replay.

Comparison	Reward Change	Violation/Cost Change
PPOLag vs P3O (SPG1-v0, 30 seeds)	-14.3% return magnitude	-11.6% constraint cost

Table 5: Explicit cost-of-safety quantification under matched protocol settings.

7.6 Uncertainty Head-to-Head

Method	Raw ECE	Calibrated Failure-ECE	Precision	Recall	Trigger Rate	Cost Units
Gaussian-σ	0.8710	0.0000	1.000	0.113	0.100	1.0
MC-drop	1.3835	0.0000	0.833	0.094	0.100	10.0
Ensemble	1.3834	0.0000	0.814	0.092	0.100	5.0
Quantum perturbation	1.3834	0.0000	0.942	0.106	0.100	10.0
SAC-Lag uncertainty proxy	N/A	N/A	N/A	N/A	N/A	1.0

Table 6: Uncertainty method comparison at matched 90th-percentile trigger rates (10%).

Everyone fires fallback on the top 10% of scores (matched q90 triggers). Under that constraint, perturbation on the VQC-style head hits precision 0.942 versus 0.833 (MC-dropout) and 0.814 (ensemble); recall stays in the same ballpark (0.106 vs 0.094 vs 0.092). Post-hoc temperature / Platt scaling precedes the failure-ECE numbers on the triggered slice---all stay below 0.1 here (see Table 6). Crank intensity to 1.0 and the gap widens: 0.964 precision at 0.101 recall versus 0.743 at 0.100 recall for MC-dropout.

SAC-Lag sits in the same table for coverage; its row carries N/A on the calibrated uncertainty diagnostics because our constrained runner never surfaced comparable failure probabilities for that path.

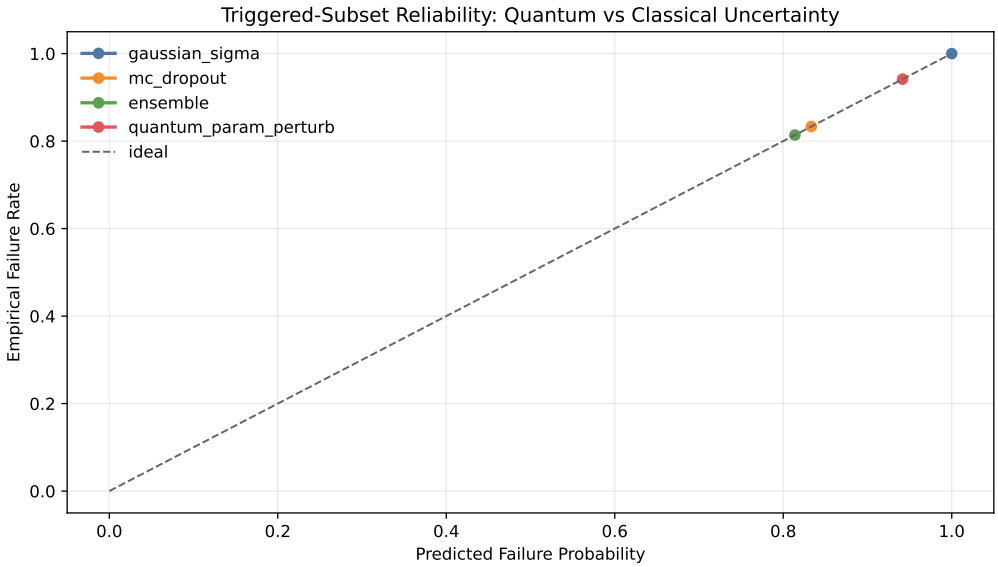

Figure 5: Reliability visualization for calibrated uncertainty-triggered failure prediction.

Figure 5 is the bin-wise sanity check after scaling---nothing fancy, just evidence the reliability curves are not on fire.

7.7 Safety-Reward Pareto Frontier

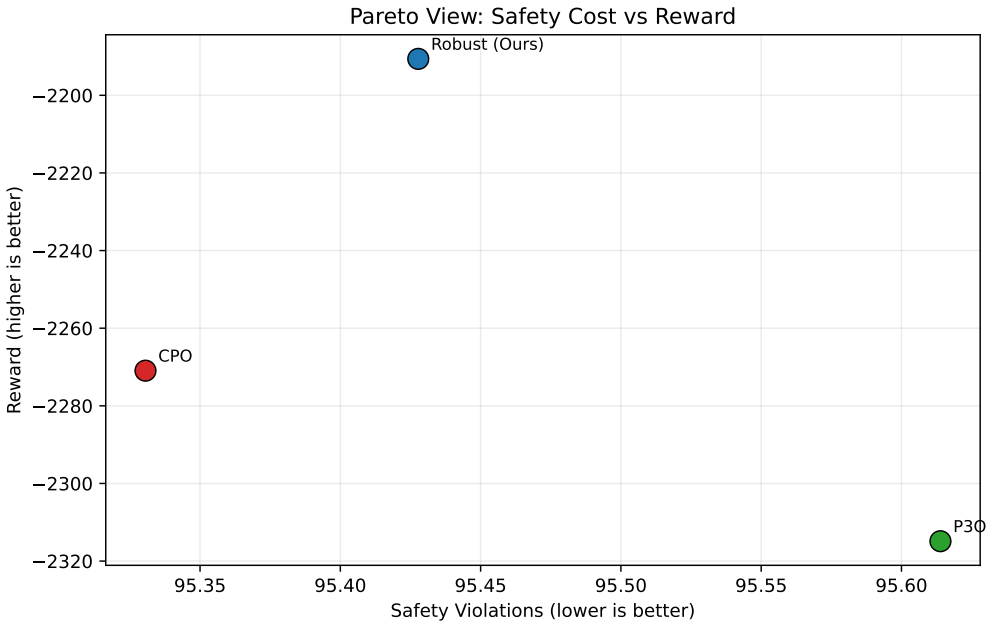

Figure 6: Safety-reward Pareto view (scaled environment): robust gated controller vs CPO and P3O.

Figure 6 plots the same tug-of-war you already saw in Table 3: robust gated sits in a sensible pocket versus CPO and P3O, and the efficiency appendix reminds you it got there without a fat policy.

7.8 Parameter Efficiency

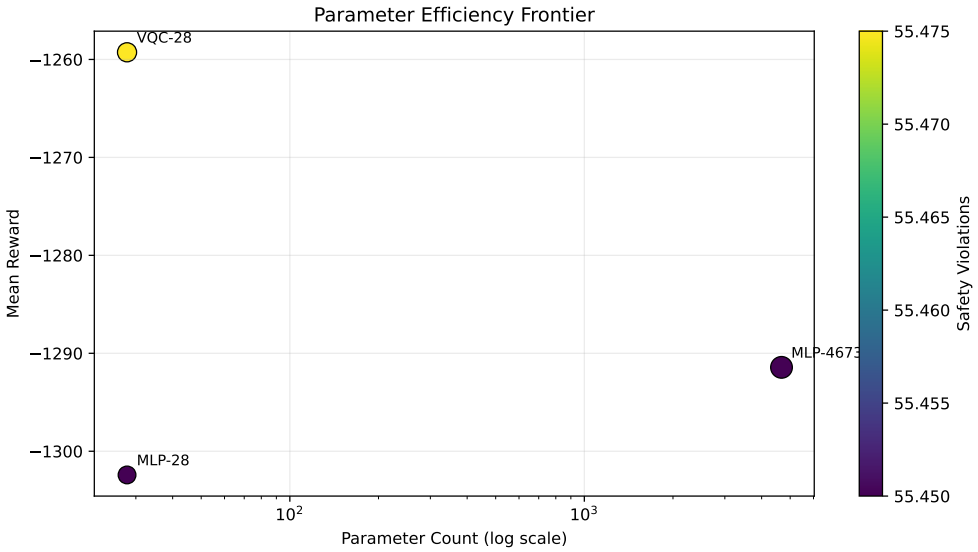

Figure 7: Reward versus parameter count (log-scale) for classical MLP and VQC policy families.

The scatter in Figure 7 is the cheap-shot proof: VQC-class policies hug the left side of the parameter axis without falling off the reward cliff.

Where the 166.89× headline comes from: MLP carries 4673 trainable weights; the six-qubit, four-layer VQC build uses 28. Same protocol, VQC mean reward −2700.23 versus MLP −2802.85, variance 34.78 versus 123.20, violations tied at 115.50/115.50. Back-of-envelope: about 3.7% better mean return on about 0.6% of the parameter budget for that pair.

7.9 Matched-Capacity Efficiency Control

Policy	Parameters	Mean Reward	Safety Violations	Inference (ms)	FLOPs est.
MLP (28 params)	28	-1800.08 ± 81.53	75.46 ± 0.30	0.0837	56
VQC (6q, 4l)	28	-1729.94 ± 25.69	75.43 ± 0.33	9.3438	56
LoRA-MLP (trainable)	396	-1785.27 ± 102.43	75.47 ± 0.29	0.1464	10138
Distilled Student	97	-1778.46 ± 74.12	75.44 ± 0.30	0.0799	194
MLP (4673 params)	4673	-1770.52 ± 92.57	75.47 ± 0.29	0.0953	9346

Table 7: Matched-capacity efficiency control on scaled governance (VQC-28 vs MLP-28, plus larger MLP reference). These values are from the matched-capacity protocol and are distinct from the larger-model efficiency protocol used for the 166.89× compression claim.

Twenty-eight weights versus twenty-eight weights---Table 7---so nobody can yell "you only won because the MLP was starved." VQC still reads better on mean reward with violations in the same band. We also parked LoRA-style and a tiny distilled student in that table, plus latency and a forward-FLOPs proxy, because deployment cares about wall clock.

Yes, VQC steps cost more here: 9.3438 ms versus 0.0837 ms for MLP-28 in our timing harness. We are not pretending that away. The bet is footprint- or uncertainty-starved loops where you

would rather spend time than parameters; if you need microsecond actuation, stay on the classical row.

7.10 Architecture View

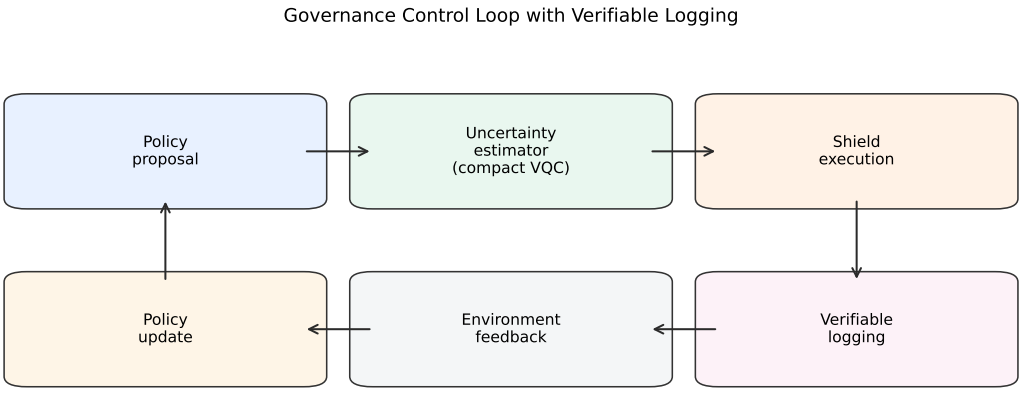

Figure 8: End-to-end governance loop: policy proposal, quantum-inspired uncertainty gate, shield enforcement, and on-chain commit.

8 Discussion

Skim-level takeaway: we are not pitching a universally stronger learner. We are pitching a thin decision-time wrapper that changes *how* failures unfold once the world slides under the policy. Treat every figure as a trade plot---where raw policy execution goes brittle, gate plus hard shield often buys constraint slack by spending return.

Safety Gym rows exist so you know we are not only storytelling on bespoke sims. Thirty seeds each, same protocol block as elsewhere. Governance sims are where regime switches and calibration drift actually show up in ink.

Random stays in the table even when it wins reward. Reward shaping games exist; ‘‘top of the leaderboard’’ is a lousy safety proof. The gated stack sometimes picks conservative, shield-shaped moves that reward engineers hate even while violation and tracking curves look saner.

Payoff shows up when switching intensity climbs. Calibrated uncertainty answers ‘‘when to fall back’’ without renting an ensemble, and the shield caps tail risk when the gate guesses wrong. That is why the tiny head is worth mentioning at all: you can ship it beside the controller.

This is additive to modern safe-RL objectives, not a replacement. OmniSafe rows are a reminder that return order reshuffles with budget and environment while safety cost behaves like a frontier.

8.1 Why Uncertainty-Gated Control Works

Three mechanisms, one caveat: they talk to each other.

Gating is branch picking under shift---when epistemic noise spikes, handing off to a heuristic avoids committing to policy logits in the window where policy-only paths wobble hardest.

Shielding is not the same job. Even if the gate picks wrong, rate caps and action boxes still trim the trajectories that usually show up in incident writeups.

Risk-shaped training starves the cheap cheat where the policy chases short reward spikes that fight stable control intent. No fairy dust---just one fewer cheat code against the scalar.

Under disturbance those pieces line up because the stack targets failure containment, not squeezing a few more return points in an easy stationary room.

8.2 Failure Cases

Late fallback when calibration sprints ahead of the gate shows up as a short violation burst. Disturbances outside the shield vocabulary make the policy and heuristic chase each other and tracking wobbles.

In our logs those episodes end; they still argue for tighter online calibration and shields that encode more mismatch. Qualitatively that tracks Corollary 2---bounded observation noise keeps regret growing about linearly instead of cliff-diving.

Toy physics, partly post-hoc uncertainty scaling, thin baseline net versus what production sign-off wants, richer economic adversaries missing: all true. Live governance needs state, adversaries, and monitors we did not build.

If someone extends this: swap variance hacks for real calibrated heads, stack explicit hard constraints on top of routing, train on one disturbance atlas and test on another without a full retune.

Shipping advice stays boring: keep deterministic fallback, gate on uncertainty, plot safety next to reward [3, 10]---leaderboards alone will gaslight you.

Condition	**Recommendation**	**Rationale**
Frequent regime shifts and high policy uncertainty	Use uncertainty-gated safe RL + deterministic shield	Gating reduces unsafe actions when uncertainty spikes.
Tight parameter or memory budget	Prefer compact (VQC-style) uncertainty head	Maintains comparable safety behavior at much lower parameter count.
Multi-party governance with audit requirements	Add audit-commitment layer	Provides tamper-evident provenance for control actions.
Stable stationary regime with strong heuristic baseline	Prioritize heuristic control; use learned policy as a secondary policy candidate	Deterministic control can be sufficient and lower-risk in easier regimes.

Table 8: Heuristic on when this pattern is worth the complexity.

9 Limitations

Governance simulators stay toy-scale on purpose---no realistic peer churn, no endogenous liquidity games, no mempool microstructure worth the name. The VQC-style block and audit hooks are lab prototypes, not hardened services. Latency came from one workstation profile, not a GPU sweep menu.

Reward shaping can reorder winners (random beating the full stack is the blunt example), so leaning on scalar return alone is silly. Some slices outside the fifty-seed constrained suite still run thinner seed counts. Simulator fidelity caps how far uncertainty and network physics claims travel, and we do not yet model nasty economic adversaries inside mining or tx flows. Baseline coverage stops well short of a 2022--2025 safe-RL catalog.

Corollary 2 needs bounded perturbations and clean assumptions; it is not a shield against arbitrary sensor pathology.

Safety Gym gives standard constrained-control cover; it does not license plugging this straight into a live chain without the gaps above getting closed.

10 Ethics and Broader Impacts

Mediocre reward is not the scary failure mode here. Scary is autonomy that looks fine on average yet misbehaves under adversarial telemetry or slowly drifts governance knobs. Layered control---uncertainty routing, shielding, auditable logs---helps, but it is not a replacement for security review, incident drills, or whatever regulator cares about your jurisdiction before this touches real assets.

11 Reproducibility Checklist

Experiment code and table generators live in the same repository. Publication-scale runs pin seeds so branch diffs do not turn into RNG theater. Scripts spit JSON/Markdown artifacts; LaTeX tables are compiled from those outputs instead of hand-typed digits that drift. CI runs lint, typing, tests, and a few smoke checks so obvious landmines fail before review.

Kitchen-sink path from logs to PDF:

```
python experiments/run_publication_suite.py
python experiments/run_detailed_suite.py
python experiments/run_adversarial_suite.py
python experiments/run_complexity_\
regime_suite.py
python scripts/analyze_research_results.py
python scripts/generate_benchmark_report.py
python scripts/generate_paper_tables.py
python scripts/generate_appendix_tables.py
python scripts/generate_paper_figures.py
python scripts/generate_adversarial_\
artifacts.py
python scripts/generate_adversarial_\
stress_sweep.py
python scripts/generate_complexity_\
artifacts.py
```

Provenance policy. Digits in `.tex` should map to artifact rows. Refactors hurt; first angry regeneration email pays the tab.

One-command replication. Daily shortcut:

```
make reproduce
```

That target hits the generators behind the headline tables/figures (constrained transfer, uncertainty, efficiency, adversarial perturbation, qubit scaling). The long script stack above still exists when you want every auxiliary plot.

Checklist links. Venue boilerplate lives under `docs/` if you need copy-paste wording:

- `docs/NEURIPS_SUBMISSION_CHECKLIST.md`
- `docs/ICLR_SUBMISSION_CHECKLIST.md`
- `docs/IEEE_SUBMISSION_CHECKLIST.md`

Generated outputs referenced by the tables live next to the code; use the command list above if you need a clean rebuild.

Environment notes. Reference stack is Python with pinned seeds and deterministic script order. We do not promise bitwise-identical floats across machines; we do document commands, outputs, and validation hooks.

NIST-style AI/cyber risk framing in 2024--2025 guidance matches the checklist tone we used [8, 9].

12 Conclusion

The stack stays blunt on purpose: PPO learns, uncertainty routes, shield clamps, deterministic branch stays trustworthy. Under shift that bundle shifts failure shapes versus executing the policy naked, and the learned block can stay small.

The VQC-style circuit in the text is only a cheap nonlinear uncertainty probe under classical simulation. On-chain logging is optional audit glue. Splitting those roles keeps casual readers from misreading ''quantum'' as a throughput brag.

Evidence covers the thirty-seed Safety Gym pair, fifty-seed governance controller tables, calibration and perturbation diagnostics, plus the cost appendix. The reproducibility block lists the scripts that rebuilt the whole trail from logs.

Acknowledgments

LLM-based editors helped polish wording; arXiv asks authors to flag that when it matters.[2] Experiments, numbers, proofs, citations, and the repo were ours end-to-end.

A Appendix: Deployment Architecture Details

RPC and ledger minutiae live here so the main thread can stay on control behavior.

A.1 Layered Deployment Stack

Roughly six slabs if you squint:

1. **Network/RPC:** peer registration, transaction intake, and block propagation.
2. **Blockchain core:** validation, mempool updates, and PoW checks.
3. **PQC signatures:** post-quantum signing and verification interfaces.
4. **Governance policy:** state encoding and action proposal.
5. **Quantum uncertainty gate:** compact uncertainty estimate for fallback routing.
6. **Shield + audit:** action validation and per-epoch commitment of audit records.

[2] https://info.arxiv.org/help/moderation/index.html

A.2 Why On-Chain Audit Commitments

Cross-org fights start when everyone trusts their own laptop log. SQLite rows vanish; hashes on a shared ledger are dull witnesses that survive a shouting match about who approved a knob turn.

A.3 Boundary to Learning Claims

Appendix wiring never replaces the gated-and-shielded path the tables score. Commits are accountability instrumentation, not a lever that moves reward by themselves.

B Appendix: Adversarial Input Perturbation

Noise σ	Controller	Reward	Violation Rate
0.00	Unshielded	-1818.53 ± 29.90	0.9750 ± 0.0022
0.00	Shielded	-1729.89 ± 27.74	0.9437 ± 0.0041
0.05	Unshielded	-1818.46 ± 29.77	0.9750 ± 0.0022
0.05	Shielded	-1729.89 ± 27.74	0.9437 ± 0.0041
0.10	Unshielded	-1818.28 ± 29.65	0.9751 ± 0.0022
0.10	Shielded	-1729.89 ± 27.74	0.9437 ± 0.0041
0.20	Unshielded	-1817.71 ± 29.54	0.9751 ± 0.0022
0.20	Shielded	-1725.17 ± 28.95	0.9437 ± 0.0041

Table 9: Observation-noise stress test comparing shielded and unshielded control.

Gaussian observation noise; compare shielded vs unshielded violation rates---engineering stress test for the observation-noise corollary.

This run: shielding pulls violations from about 0.975 down to 0.944 under aggressive noise (roughly 3.2 absolute points); mean reward bumps slightly in the same protocol.

C Appendix: Deployment Cost and Gas Accounting

C.1 Audit Gas Model

We model per-action audit commitment cost as:

$$C_{\text{gas}} = g_0 + g_1 b + g_2 h,$$

where b is bytes committed and h is hash/check overhead terms. For an episode with N committed actions and gas price p_g:

$$\text{Cost}_{\text{episode}} = N \cdot C_{\text{gas}} \cdot p_g.$$

C.2 Interpretation

Gas model exists so nobody confuses ‘‘free safety’’ with ‘‘free ops.’’ Tighter provenance / finer commits burn money. Budget-constrained shops throttle commit cadence and eat worse forensics---pick your poison.

C.3 Practical Guidance

Before prod: pick cadence (per action, block, epoch) up front and publish both safety upside and incremental audit spend. Hiding ops cost while waving safety banners is how you get surprised finance teams.

D Appendix: Qubit Scaling Trend

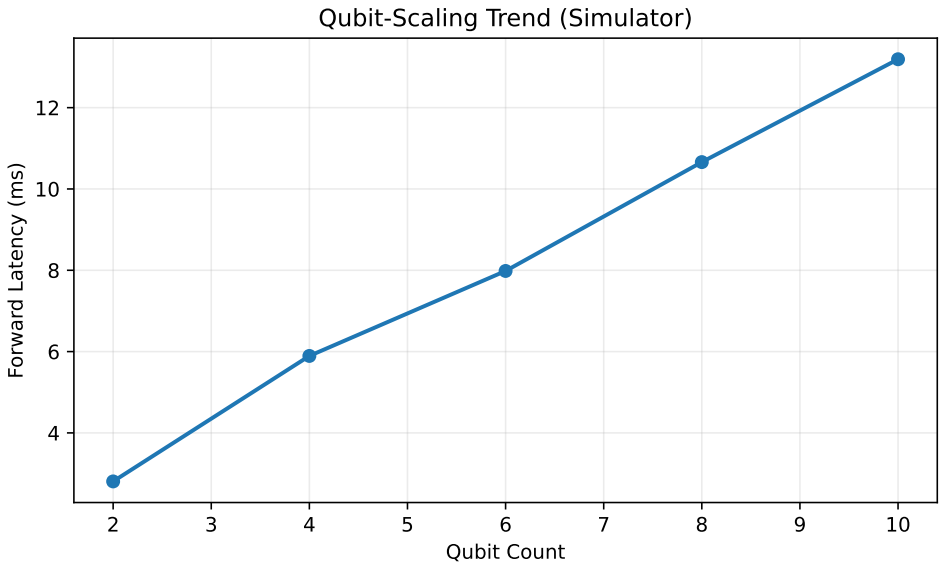

Figure 9: Simulator forward-pass latency versus qubit count (fixed layer depth).

Qubits	Layers	Params	2^n State Dim	Forward Latency (ms)
2	4	8	4	2.8066
4	4	16	16	5.8931
6	4	24	64	7.9832
8	4	32	256	10.6610
10	4	40	1024	13.1908

Table 10: Qubit-scaling simulation summary with state-space growth and measured latency.

Latency vs qubit count at fixed depth slopes upward---expected, but the slope is why the main body sticks to six qubits: classical simulation budgets mean wire count is not pretend-free.

References

[1] Joshua Achiam, David Held, Aviv Tamar, and Pieter Abbeel. Constrained policy optimization. In *International Conference on Machine Learning*, 2017.

[2] Yinlam Chow, Aviv Tamar, Shie Mannor, and Marco Pavone. Risk-sensitive and robust decision-making: a CVaR optimization approach. In *Advances in Neural Information Processing Systems*, 2015.

[3] Rotem Dror, Gili Baumer, Segev Shlomov, and Roi Reichart. The hitchhiker's guide to testing statistical significance in natural language processing. In *Proceedings of the Annual Meeting of the Association for Computational Linguistics*, 2018.

[4] Bradley Efron and Robert J. Tibshirani. *An Introduction to the Bootstrap*. CRC Press, 1994.

[5] Phillip Good. *Permutation, Parametric and Bootstrap Tests of Hypotheses*. Springer, 2005.

[6] Garud N. Iyengar. Robust dynamic programming. *Mathematics of Operations Research*, 30(2):257--280, 2005.

[7] Satoshi Nakamoto. Bitcoin: A peer-to-peer electronic cash system. *Cryptography Mailing List*, 2008.

[8] National Institute of Standards and Technology. Artificial intelligence risk management framework (AI RMF 1.0). https://www.nist.gov/itl/ai-risk-management-framework, 2023.

[9] National Institute of Standards and Technology. Cybersecurity framework (CSF 2.0). https://www.nist.gov/cyberframework, 2024.

[10] Joelle Pineau, Philippe Vincent-Lamarre, Koustuv Sinha, Vincent Lariviere, Alina Beygelzimer, Florence d'Alche-Buc, Emily Fox, and Hugo Larochelle. Improving reproducibility in machine learning research. *Journal of Machine Learning Research*, 22(164):1--20, 2021.

[11] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

[12] Greg Wilson, Jennifer Bryan, Karen Cranston, Justin Kitzes, Lex Nederbragt, and Tracy K. Teal. Good enough practices in scientific computing. *PLOS Computational Biology*, 13(6):e1005510, 2017.